NAZARBAYEV UNIVERSITY

Programming for Engineers

Comprehensive Course Review

Self-Study and Practice | Weeks 1–13

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026



NAZARBAYEV UNIVERSITY

How to use this review independently

1. Choose a topic from the linked contents or PDF bookmarks.
2. Read the recap and trace each commented example.
3. On a question page, write a prediction before opening the answer.
4. Compare the reasoning, not just the final output.
5. Run the supplied example, change an input, and test your explanation.

No timetable is imposed

This is a topic-organized review, not three lectures. Continue at your own pace and return to weak areas. Every practice question has a visible worked-answer page in this complete deck.

Contents

NAZARBAYEV UNIVERSITY

Contents | Foundations and dependable code

Module	Topic	Original week
01 – Open module	Computation, variables, and types	1
02 – Open module	Decisions and repetition	2
03 – Open module	Strings and numerical search	3
04 – Open module	Functions and program design	4
05 – Open module	Lists, tuples, and data integrity	5
06 – Open module	Recursion and dictionaries	6
07 – Open module	Testing, debugging, and failure	7

Continue to later topics

- Classes, algorithms, languages, and practice

NAZARBAYEV UNIVERSITY

Contents | Objects, algorithms, and practice

Module	Topic
08 – Open module	Classes, instances, and methods
09 – Open module	Interfaces, shared state, inheritance
10 – Open module	Efficiency: counts and growth
11 – Open module	Hidden work, recursion, and memory
12 – Open module	Searching and sorting
13 – Open module	Programming beyond Python
14 – Open module	Integrated engineering review
15 – Open module	Mixed-topic practice
16 – Open module	Final checks and reference

Choose a study route, not a fixed pace

Full review

Work through Modules 01–13 in the course order, then the integrated example and mixed-topic practice.

Targeted repair

Attempt a relevant question first. If the prediction or explanation fails, return to that module's recap, example, and tests.

Scope and assumptions

The review follows the instructor's actual sequence. It introduces no new assessed topic or assignment requirement. Illustrative thresholds and material data are not design standards.

Contents

NAZARBAYEV UNIVERSITY

The answers are part of the student resource

- Question IDs remain the same in the complete deck, practice-only PDF, and answer index.
- A Worked answer button jumps to the solution; a Return to question button takes you back.
- Answers are printed on ordinary PDF pages, not hidden in presentation notes.
- A separate questions-only PDF supports a first attempt without accidentally seeing the next solution.

Keep independent evidence

Record your own trace and expectation before running code. A correct final output with an incorrect explanation is a reason to revisit the concept.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

01 | Computation, variables, and types

Week 1: turn a question into a checkable calculation

NAZARBAYEV UNIVERSITY

A computational result is an engineering argument

The computational chain

1. State the question and required output.
2. Identify inputs, units, and assumptions.
3. Write an algorithm; implement it in Python.
4. Test independently; interpret the result.

Keep these ideas distinct

- A **model** represents the situation.
- An **algorithm** is a procedure.
- A **program** implements a procedure.
- An **output** is evidence to interpret.

Course context

ENG 101 – 2L, Programming for Engineers, 6 ECTS; Fall 2026. Instructor: Yanwei Wang. Use Moodle for current assessment instructions.

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Arithmetic: the operator and the operands both matter

Operation	Python	Result / interpretation
Add, subtract, multiply	`+`, `-`, `*`	Use explicit multiplication; no implicit 2x.
True division	`7 / 2`	`3.5`; division of ordinary integers returns a float.
Floor division	`7 // 2`	`3`; floor the quotient, not ordinary rounding.
Remainder	`7 % 2`	`1`; useful for divisibility and cycles.
Power	`2 ** 3`	`8`; `^` is not Python exponentiation.

Translate the formula first

$E_k = \frac{1}{2}mv^2$ becomes `0.5 * mass_kg * velocity_m_s**2`. Parentheses should make grouping visible.

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Assignment is an event, not a permanent equation

```
# Evaluate the right-hand side at each
    assignment.
radius = 2.0
area = 3.14 * radius**2
radius = 3.0
print(radius, area)
```

Trace the stored result

Changing `radius` does not rerun the earlier area calculation. The name `area` still refers to the previously computed value 12.56.

Naming habits

Use meaningful names, consistent case, and visible units. `area_mm2` and `area_m2` are different quantities, not interchangeable labels.

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Types describe values; appearance does not

Value	Type	Useful interpretation
`12`	`int`	A count or exact integer.
`12.0`	`float`	An approximate numerical value.
`"12"`	`str`	Text: it may be an identifier.
`True`	`bool`	An explicitly stated yes/no condition.
`None`	`NoneType`	No result or absent information under a contract.

Input and output

`input()` returns text when it succeeds. Convert only when the intended meaning is numerical. `print()` displays a result; it does not establish its correctness.

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A complete unit conversion: force and area

```
# Uniform axial stress; N and mm^2 are
    supplied.
force_n = 10_000.0
area_mm2 = 500.0
area_m2 = area_mm2 * 1e-6
stress_pa = force_n / area_m2
stress_mpa = stress_pa / 1e6
print(f"Stress = {stress_mpa:.1f} MPa")
```

Independent calculation

$500\,\mathrm{mm}^2 = 5 \times 10^{-4}\,\mathrm{m}^2$.
$\sigma = 10\,000/(5 \times 10^{-4}) = 20\,\mathrm{MPa}$.

Model scope

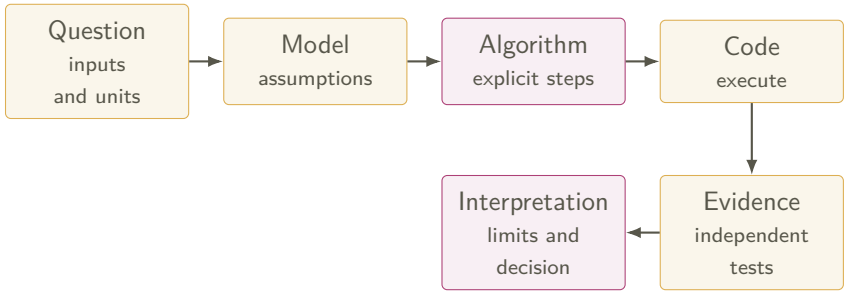
The example assumes uniform axial stress. A result within a stated limit is not a complete assessment of structural safety.

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Keep the entire engineering chain visible



Two claims, two kinds of evidence

The code implements the model. The model is appropriate for the situation. Neither claim automatically establishes the other.

Contents

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

NAZARBAYEV UNIVERSITY

M01-Q1 | Values, types, and precedence

```
# Predict exact outputs before executing
    this example.
print("7" + "2", int("7") + 2)
print(7 / 2, 7 // 2, 7 % 2)
print("3" * 2, 3 * 2)
print(2 + 3 * 4, (2 + 3) * 4)
```

Write your prediction first

1. Write all four printed lines.
2. Which line repeats characters instead of multiplying a number?
3. Which grouping changes the result?

▸ Worked answer

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

M01-Q1 | Worked answer

```
72 9
3.5 3 1
33 6
14 20
```

Why this happens

`"7" + "2"` joins text. `int("7")` converts it. True division and floor division differ. Multiplication is evaluated before addition unless parentheses change the grouping.

Return to question

Contents

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

NAZARBAYEV UNIVERSITY

M01-Q2 | A unit error can look plausible

Try before viewing the solution

A 2 kW heater operates for 30 minutes. A program evaluates `energy_kwh = 2 * 30`. Explain the defect, calculate the correct kWh value, and state one assumption.

▸ Worked answer

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

M01-Q2 | Worked answer

Worked reasoning

Power in kW must be multiplied by time in hours for a kWh result. Thus $E = 2(30/60) = 1$ kWh. The expression gives 60 kW min, not 60 kWh. We assume constant 2 kW input power; the calculation does not specify useful-heat efficiency.

Return to question

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Week 1 checkpoint

- Explain the difference between a problem, model, algorithm, and program.
- Trace reassignment without imagining that old expressions update automatically.
- Distinguish a string, integer, float, Boolean, and absence marker.
- Translate a formula and derive its units before trusting the output.

Before moving on

Rerun a known-value calculation after changing one input. Predict both its numerical trend and its physical unit before execution.

[W01] computation, Python basics, operations, variables, and types. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

02 | Decisions and repetition

Week 2: specify the branch and the stopping rule

NAZARBAYEV UNIVERSITY

One branch or several independent decisions?

A single classification

An `if/elif/else` chain selects the **first true** condition. Later branches in that chain are skipped.
For increasing thresholds, test the lower categories before wider ranges.

Independent checks

Several separate `if` statements may all run. This is useful when more than one warning is allowed.

Indentation is part of the algorithm

An indented statement belongs to its enclosing block. Moving a report line outside a branch can print a warning even when its condition is false.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Boolean logic should read like the rule

Intent	Expression	Boundary meaning
Both conditions hold	`temperature_ok and pressure_ok`	Both individual checks are required.
Either warning exists	`too_hot or too_high`	One true warning is sufficient.
Reverse a Boolean	`not connected`	The device is not connected.
Inclusive interval	`20 <= value <= 30`	Both 20 and 30 are accepted.
Recognized unit	`unit in ("pa", "kpa")`	Compare to each permitted string.

Classic defect

`unit == "pa" or "kpa"` does not perform two equality tests: a nonempty string is truthy.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Boundaries are not optional details

```
# The stated classroom rule accepts FoS
    exactly 2.
fos = 2.0
if fos < 1.0:
    status = "failure expected"
elif fos < 2.0:
    status = "insufficient margin"
elif fos < 4.0:
    status = "acceptable by stated rule"
else:
    status = "conservative"
print(status)
```

Reason through the chain

At 2.0, the first two tests are false. The third is true. Test values just below, exactly at, and just above each threshold.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

range defines candidates; control flow defines visits

Expression	Generated integers
`range(4)`	0, 1, 2, 3
`range(1, 5)`	1, 2, 3, 4
`range(2, 9, 2)`	2, 4, 6, 8
`range(5, 0, -2)`	5, 3, 1
`range(3, 3)`	No values; the loop body is skipped.

Three reminders

The stop value is excluded. The step cannot be zero. A `break` can stop execution before every generated value is visited.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Counters, accumulators, and system state

Pattern	Initialization	Update
Count warning events	`count = 0`	`count += 1` only on a warning.
Sum loads	`total_n = 0.0`	`total_n += load_n` each included load.
Track elapsed time	`time_s = 0.0`	`time_s += step_s` each step.
Track a filled volume	`volume_l = 0.0`	`volume_l += flow_l_s * step_s`.

Initialization belongs before the loop

Resetting an accumulator inside the loop destroys the history. A counter counts events; it is not automatically a physical total.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A while loop needs progress

```
# Stop on success or a finite budget.
attempts = 0
while error > tolerance and attempts <
    maximum:
    # Update the candidate and error.
    candidate = candidate + step
    error = abs(candidate**2 - target)
    attempts += 1
# Check the stopping reason afterward.
```

Ask four questions

1. Is the condition false initially?
2. What changes on each pass?
3. Can the update reach a stopping state?
4. Did the loop stop from success or from its budget?

Snippet status

This is a pattern, not a standalone program: the target, initial error, step, tolerance, and budget must be defined.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M02-Q1 | Trace break and continue

```
# Track both the visited value and the
    accumulated total.
total = 0
for value in range(1, 7):
    if value == 2:
        continue
    if value == 5:
        break
    total += value
print(total)
```

Write your prediction first

1. Write the final total.
2. List the visited loop values.
3. Explain whether value 6 is processed.

▸ Worked answer

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

M02-Q1 | Worked answer

8

Why this happens

The visits are 1, 2, 3, 4, and 5. The totals are 1, 1, 4, 8, and 8. At 2, continue skips the remaining body; at 5, break ends the loop before addition. Value 6 is never visited.

Return to question

Contents

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

NAZARBAYEV UNIVERSITY

M02-Q2 | Repair a classification

Try before viewing the solution

The rule accepts $20 \leq T \leq 30$. Explain why testing `if T < 30: "normal"` before `elif T < 20: "too low"` fails. Give three boundary tests.

▸ Worked answer

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

M02-Q2 | Worked answer

Worked reasoning

The first branch already accepts values below 20, so the later low-temperature branch cannot handle them. Use `if T < 20`, then `elif T <= 30`, then `else`. Tests: 19.9 gives too low; 20.0 and 30.0 give normal. Also test 30.1 for the high branch.

Return to question

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M02-Q3 | Stopping is not exact event timing

Try before viewing the solution

A tank begins with 0 L and fills at 3 L/s. A loop adds 3 L and 1 s each pass while volume is below 10 L. What final state is reported? What is the exact constant-flow filling time?

▸ Worked answer

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

M02-Q3 | Worked answer

Worked reasoning

After 4 passes the state is 12 L at 4 s. The exact time is $10/3 \approx 3.3333$ s under constant flow. The full-step simulation first observes the target at 4 s; its event time is approximate. Clipping 12 L to 10 L without adjusting time does not make the event time exact.

Return to question

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Week 2 checkpoint

- Convert the English rule into explicit comparisons.
- Choose a chain or independent checks deliberately.
- Trace changes in state rather than only the last printed value.
- Test zero, one, and several loop passes; identify the stopping route.

Learning test

Explain why a program can execute without a Python error and still implement the wrong rule.

[W02] branching, Boolean expressions, indentation, for/range/while. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

03 | Strings and numerical search

Week 3: preserve meaning and justify approximations

NAZARBAYEV UNIVERSITY

Indexing versus slicing

Expression for s = "PUMP-107A"	Result	Rule
`s[0]`	`"P"`	First item: index zero.
`s[-1]`	`"A"`	Last item.
`s[:4]`	`"PUMP"`	Start omitted; stop excluded.
`s[5:]`	`"107A"`	Continue to the end.
`s[::2]`	`"PM-0A"`	Indices 0, 2, 4, 6, 8.
`s[::-1]`	`"A701-PMUP"`	Reverse order.

Explicit correction to an earlier example

The Week 3 shortcut table printed `"PMP0A"` for `s[::2]`. Executing the displayed string gives `"PM-0A"`. The selected indices show why.

Contents

[W03] PDF p. 11; slicing-result correction is explicitly identified and executed in the checks.

NAZARBAYEV UNIVERSITY

String transformations return new text

```
# Preserve the raw record; name each
    transformation.
raw = " Pump-007 "
clean = raw.strip()
upper = clean.upper()
has_prefix = upper.startswith("PUMP-")
position = upper.find("-")
pieces = upper.split("-")
joined = ":".join(pieces)
```

Interpret the returned value

- `strip()` removes surrounding whitespace.
- `find()` returns an index or -1.
- `split()` returns pieces.
- `join()` combines string pieces.

Preserve identity

A leading zero or punctuation can identify an asset. Normalize only under a documented rule; a unit label is not an automatic conversion.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Parsing requires a record contract

```
# Assumption: exactly three comma-separated
    fields.
raw = " T-07 , 23.50 , C "
name, text, unit = raw.split(",")
name = name.strip()
value = float(text)
unit = unit.strip()
print(name, value, unit)
print(name[-2:])
```

Do not skip validation

Unpacking expects three fields. A numeric conversion can fail. The expected unit must be checked separately. Week 7 tools let the program report these failures explicitly.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Guess-and-check is systematic

Part	Question
Candidate	Which value will be tested next?
Evaluation	What does the model predict for this candidate?
Acceptance	What exact result or tolerance is required?
Update	How does the next candidate differ?
Stopping safeguard	What finite range or work budget prevents endless work?

Exact cube-root example

For an integer target, enumerate nonnegative magnitudes until `candidate**3 >= abs(target)`. Then test equality and restore the sign for a negative perfect cube. Passing the target means no integer cube root was found.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Incremental search: step size is not tolerance

Resolution

A step h selects a grid of candidate values. A small residual tolerance ε_f selects which candidates are acceptable. They are different parameters.

Cost and risk

Searching an interval of length L takes about L/h trials. A coarse grid may skip every acceptable value even when an exact solution lies in the interval.

Evidence to report

Report the candidate, $|f(x)|$, chosen tolerance, number of trials, and stopping reason. Exhausting the budget must not be reported as convergence.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Bisection preserves a bracket

Required reasoning

- Express the target as $f(x) = 0$.
- Assume f is continuous on $[a, b]$.
- Handle exact endpoint roots.
- Otherwise require opposite endpoint signs.

One update

1. Compute $m = (a + b)/2$.
2. Inspect the midpoint residual.
3. Retain the half with opposite endpoint signs.
4. Repeat while the chosen accuracy test is unmet.

Why the method is useful

Each exact-arithmetic update halves a bracket that still contains a root.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A bisection trace gives a checkable argument

a	b	m	f(m) = m^2 - 10	Retain
3	4	3.5	2.25	[3, 3.5]
3	3.5	3.25	0.5625	[3, 3.25]
3	3.25	3.125	-0.234375	[3.125, 3.25]
3.125	3.25	3.1875	0.16015625	[3.125, 3.1875]

Check the invariant

The negative residual stays at the left endpoint and the positive residual at the right in this example. The new bracket is not chosen by which endpoint is numerically smaller than zero.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Two tolerances answer two different questions

Residual tolerance

$|f(x)| \le \varepsilon_f$ measures agreement with the **output target**. Its unit is the residual unit. Small residual alone need not imply small input error.

Input-space bound

For midpoint m of a valid bracket, $|m - x^*| \le (b-a)/2$ in exact arithmetic. Its unit is the **input variable** unit.

Count the halvings

After k completed halvings, $L_k = L_0/2^k$. To guarantee midpoint error at most ε_x, require $L_0/2^{k+1} \le \varepsilon_x$. State whether your tolerance means width or half-width.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

What bisection cannot infer from two samples

- A sign change across a discontinuity is not a root certificate: $1/x$ changes sign across zero but has no zero there.
- A touching root may have no sign change: $(x-1)^2 = 0$ at 1.
- Two crossings can leave the endpoint signs equal.
- A finite iteration limit or floating-point stagnation is not numerical success.

Model validity is separate

Even a converged root of a cooling or projectile model inherits that model's assumptions. Check the physical model and units as well as the iteration.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Run ten bracket-preserving updates

```
# Specific root x^2 = 10; [3, 4] is a valid initial bracket.
left, right = 3.0, 4.0
for _ in range(10):
    mid = (left + right) / 2
    if mid**2 < 10:
        left = mid
    else:
        right = mid
estimate = (left + right) / 2
print("Estimate, half-width:", estimate, (right - left) / 2)
```

What is guaranteed here?

Ten halvings give width $1/1024$ and midpoint error bound $1/2048$ in exact arithmetic. This specializes the general sign-bracket idea to the increasing function $x^2 - 10$ on [3,4].

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Halving turns a wide bracket into evidence



Read this as uncertainty reduction

Each horizontal segment is a retained interval, not a sequence of measured errors. The exact root shown is an independent reference for this particular example.

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M03-Q1 | Trace strings without changing the original

```
# Each transformed value is assigned to a
    separate name.
raw = " valve-204b "
code = raw.strip().upper()
print(code[:5], code[6:9], code[-1])
print(code[1:8:2])
print(repr(raw))
```

Write your prediction first

1. Write the three output lines.
2. Explain whether raw is modified.
3. Which characters does [6:9] select?

▸ Worked answer

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

M03-Q1 | Worked answer

```
VALVE 204 B
AV-0
' valve-204b '
```

Why this happens

`strip()` and `upper()` return new strings. The original raw value retains its spaces and case. Indices 6, 7, and 8 in the cleaned ID contain 2, 0, and 4.

Return to question

Contents

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

NAZARBAYEV UNIVERSITY

M03-Q2 | Turn bracket width into an error bound

Try before viewing the solution

A valid bracket has initial width 1. After 8 completed halvings, what is its width and what is the midpoint error bound? How many halvings suffice for midpoint error at most 10^{-3}?

▸ Worked answer

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

M03-Q2 | Worked answer

Worked reasoning

The width is $2^{-8} = 0.00390625$; the midpoint error bound is half of that, 0.001953125. After 9 halvings the bound is $2^{-10} = 0.0009765625 \leq 10^{-3}$. This assumes the bracket invariant, continuity, and exact arithmetic.

Return to question

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M03-Q3 | No acceptable grid point is not no root

Try before viewing the solution

Search candidates 3.0, 3.1, 3.2, ..., 4.0 for $|x^2 - 10| < 0.001$. Explain why this search fails despite $\sqrt{10} \in [3, 4]$. Propose a better approach.

▸ Worked answer

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

Contents

M03-Q3 | Worked answer

Worked reasoning

At 3.1 the residual is -0.39; at 3.2 it is 0.24. The acceptable region lies between grid points. Failure describes this candidate grid, not the existence of a root. Bisection on [3,4] exploits a sign-changing continuous bracket; its accuracy can be controlled by bracket width.

Return to question

Contents

[W03] string manipulation, guess-and-check, approximate solutions, bisection. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

04 | Functions and program design

Week 4: clear interfaces connect independently tested parts

NAZARBAYEV UNIVERSITY

Define a tool; then call it

```
# Inputs: force in N and positive area in m
    ^2.
def calculate_stress(force_n, area_m2):
    """Return uniform axial stress in Pa."""
    return force_n / area_m2
# A call evaluates arguments and executes
    the function.
result_pa = calculate_stress(10.0, 2.0)
```

Anatomy of the call

1. Evaluate argument expressions.
2. Bind them to parameter names.
3. Execute the indented body.
4. Return a value and continue at the caller.

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Parameters, arguments, and defaults

Concept	Example	Meaning
Parameter	`force_n` in the header	A local name used during the call.
Argument	`10.0` in a call	A supplied value.
Positional call	`f(10.0, 2.0)`	Order determines binding.
Keyword call	`f(area_m2=2, force_n=10)`	Names determine binding.
Default	`required_fos=2.0`	A documented choice used if omitted.

Meaning remains the caller's responsibility

Correct argument types do not enforce correct units. A default is not a universal engineering standard.

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Return values, displayed output, and side effects

Action	Caller receives	What to inspect
`return value`	The value; the call ends.	Its type, unit, and correctness.
`print(value)`	`None` from print.	Text is displayed, not returned as data.
No executed return	`None` from the function.	Did the calculation need a reusable result?
Modify an argument	Possibly a changed shared object.	A side effect is separate from the return.

Design pattern

Calculation functions return results. Reporting functions construct text. The caller chooses whether to print, save, or further process the result.

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Scope: local names are not caller variables

```
# This local assignment does not rebind the
    outer name.
value = 10
def double(value):
    value = 2 * value
    return value
result = double(value)
print(value, result)
```

What crosses the interface?

The caller supplies an object through an argument and receives a returned result. Its variable name is not transferred. A mutable argument can still be changed through a shared reference.

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Decomposition and abstraction are complementary

Decomposition

Break the program into focused responsibilities: parse, validate, convert, calculate, classify, and report. Test each part, then test their connections.

Abstraction

Use a component through its name and contract without reconstructing every internal step. This works only when the promise is clear and tested.

A useful engineering interface

State input meanings and units, valid ranges, output meaning and accuracy, mutation behavior, and expected failures. A docstring records the promise; it does not enforce it.

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Write the specification first

Component	Input contract	Output contract
Area conversion	Finite positive area in mm^2	Same area in m^2.
Stress calculation	Force in N; positive area in m^2	Signed uniform axial stress in Pa.
Classification	Value and ordered limits in one unit	Category with an explicit boundary rule.
Report	Result plus units and assumptions	Human-readable text, without changing data.

Test the interface chain

A passing conversion test and a passing stress test do not prevent a caller from passing the wrong unit between them. Keep an end-to-end test: 10,000 N and 500 mm^2 must give 20 MPa.

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M04-Q1 | What does the caller store?

```
# Intentional design defect: display without
      a return.
def energy(power_w, time_s=2):
    energy_j = power_w * time_s
    print(energy_j)
result = energy(10)
print(result)
```

Write your prediction first

1. Predict both printed lines.
2. Identify the default parameter value.
3. How should the function be repaired for later arithmetic?

▸ Worked answer

Contents

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

M04-Q1 | Worked answer

```
20
None
```

Why this happens

The function displays 20 but returns `None`. The default time is 2 s. Replace the internal print with `return energy_j`; then display the result in the caller.

Return to question

Contents

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

NAZARBAYEV UNIVERSITY

M04-Q2 | Expose a hidden dependency

Try before viewing the solution

A global `area_m2 = 0.002` is read inside `def stress(force_n): return force_n / area_m2`. Why is that interface incomplete? Give a better header and one independent test.

▸ Worked answer

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M04-Q2 | Worked answer

Worked reasoning

The result depends on an area absent from the parameter list. Use `def stress(force_n, area_m2)` and return their ratio. With force 10 N and area 2 m^2 the expected result is 5 Pa. Document the positive-area precondition and test rejection if the implementation promises validation.

Return to question

Contents

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

NAZARBAYEV UNIVERSITY

M04-Q3 | Repair an early return

```
# Intentional logic defect: the return is
    inside the loop.
def total_load(loads_n):
    total = 0.0
    for load in loads_n:
        total += load
        return total
print(total_load([2.0, 3.0, 4.0]))
```

Write your prediction first

1. Predict the printed result.
2. Explain which values never contribute.
3. Where should return be placed?

▸ Worked answer

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

Contents

M04-Q3 | Worked answer

```
2.0
```

Why this happens

The first pass adds 2.0 and immediately returns it. Values 3.0 and 4.0 are not processed. Put `return total` after the loop, at the loop header's indentation level; the corrected result is 9.0.

Return to question

Contents

[W04] functions, parameters, return values, scope, decomposition, abstraction. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

05 | Lists, tuples, and data integrity

Week 5: store order and control mutation

NAZARBAYEV UNIVERSITY

Choose a sequence from what its order means

List

An ordered, mutable collection: use it for measurements, changing candidate sets, or a sequence built gradually.

Tuple

An ordered collection with fixed element references: use it for a short record such as coordinates or paired measurements.

Neither automatically stores units

Record units in names and documentation. Sorting a time series changes its order; sorting one of two paired lists alone destroys the pairing.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Inspect length, position, and selected windows

Expression	Interpretation	Important boundary
`len(values)`	Number of elements.	An empty list has length zero.
`values[i]`	One item by index.	For length n, $0 \leq i < n$.
`values[-1]`	Last element.	Invalid for an empty list.
`values[a:b]`	New shallow slice.	Stop excluded; out-of-range bounds are clipped.
`enumerate(values)`	Index and value together.	Default index begins at zero.

Slice length needs a qualification

For ordinary bounds $0 \leq a \leq b \leq n$ and step 1, length is $b - a$. That shortcut does not apply unchanged to clipped, reversed, or stepped slices.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

List operations: ask two separate questions

Operation	Changes this list?	Returns
`a.append(x)`	Yes: add one object.	`None`
`a.extend(xs)`	Yes: add each supplied element.	`None`
`a.remove(x)`	Yes: first matching value.	`None`; error if absent.
`a.pop(i)`	Yes: remove at position i.	The removed element.
`a.sort()`	Yes: reorder in place.	`None`
`sorted(a)`	No.	A new sorted list.

Common failure

`a = a.sort()` rebinds a to `None`; it does not store the sorted list.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A copied outer list can still share inner objects

```
# A shallow copy creates a new outer list
    only.
raw = [[1, 2], [3, 4]]
copied = raw.copy()
copied[0][0] = 9
print(raw)
print(copied is raw, copied[0] is raw[0])
```

Draw the references

The two outer lists are distinct. Their first elements refer to the same inner list. Replacing an inner item changes the shared inner object.

Choose copying depth deliberately

For flat numeric data a shallow copy separates element replacement. For nested mutable data, decide which contained objects must also be independent.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Filtering should not skip shifted elements

```
# Build a new list; do not remove from the
    iterated list.
raw = [1, 2, 2, 3, 2, 4]
cleaned = []
for value in raw:
    if value != 2:
        cleaned.append(value)
# cleaned is [1, 3, 4]; raw remains
    available.
```

Why removing during iteration is risky

Deletion shifts later elements left while the iterator advances. Adjacent matches can be skipped. A new result makes the inclusion rule and raw-data preservation visible.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Tuple syntax and the limits of immutability

```
# Commas establish tuple structure.
empty = ()
singleton = (4.0,)
point = (2.0, 5.0)
x_m, y_m = point
# A list of fixed-shape records preserves
    each pairing.
samples = [(0.0, 20.0), (1.0, 22.0)]
for time_s, temperature_c in samples:
    print(time_s, temperature_c)
```

What cannot change?

The tuple's slots cannot be replaced in place. An object referred to by a slot may itself be mutable; tuple immutability is not recursive immutability.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A numerical summary needs a data contract

Compute deliberately

For $n > 0$, $\bar{x} = \sum x_i / n$ and $x_{\mathrm{RMS}} = \sqrt{\sum x_i^2 / n}$. Use the actual count of included readings.

Interpret deliberately

- What happens for empty data?
- Were missing or rejected records removed explicitly?
- Are all values in the same unit?
- Does the summary hide transients or order?

Do not fill missing data with zero silently

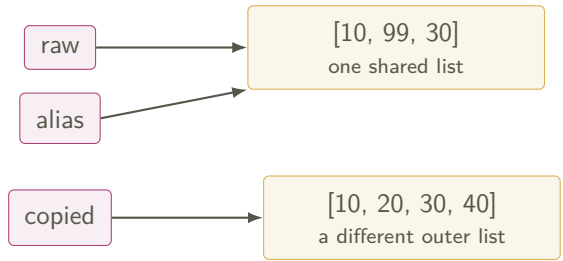
Zero is a measurement value, not a universal marker for missing information.

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Draw the reference graph to explain mutation



Track assignment targets

A name can be rebound without changing an object. Assigning a list element changes the object at that position. A shallow copy changes the outer reference graph, not necessarily the nested objects.

Contents

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

NAZARBAYEV UNIVERSITY

M05-Q1 | Trace operations and their return values

```
# Record object state and returned results
    separately.
values = [10, 20, 30]
values[1] = 25
result = values.append(40)
values.extend([50, 60])
removed = values.pop()
print(values)
print(result)
print(removed)
```

Write your prediction first

1. Write the three output lines.
2. Explain the length after extend.
3. Which operation returns None?

▸ Worked answer

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M05-Q1 | Worked answer

```
[10, 25, 30, 40, 50]
None
60
```

Why this happens

The replacement makes 25 the middle item. Append adds one 40; extend adds two items, 50 and 60. Pop removes and returns 60. The return from append is `None`.

Return to question

Contents

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

NAZARBAYEV UNIVERSITY

M05-Q2 | Alias or independent outer list?

```
# Compare aliasing with a shallow copy of
    flat numeric data.
raw = [10, 20, 30]
alias = raw
copied = raw.copy()
alias[1] = 99
copied.append(40)
print(raw)
print(alias)
print(copied)
```

Write your prediction first

1. Predict the three lists.
2. Which assignment creates an alias?
3. Why does appending not affect raw?

▸ Worked answer

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M05-Q2 | Worked answer

```
[10, 99, 30]
[10, 99, 30]
[10, 20, 30, 40]
```

Why this happens

`alias = raw` binds a second name to the same list. The item replacement changes that shared list. The copy was made earlier, so it retains 20 and receives its own appended 40.

Return to question

Contents

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

NAZARBAYEV UNIVERSITY

M05-Q3 | Preserve paired measurements

Try before viewing the solution

Time values are [0, 1, 2] s and temperatures are [25, 20, 30] °C. Why is sorting only temperatures misleading? Give a safer representation and one boundary check for a rate calculation.

▸ Worked answer

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

M05-Q3 | Worked answer

Worked reasoning

Separate sorting falsely associates 20 °C with 0 s instead of 1 s. Store [(0,25),(1,20),(2,30)] as records and sort whole records only for a stated purpose. A rate between records requires a nonzero time difference and an explicit decision about repeated or decreasing time stamps.

Return to question

[W05] lists, tuples, indexing, ordering, mutation, and list operations. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

06 | Recursion and dictionaries

Week 6: reduce a problem and preserve meaning by key

NAZARBAYEV UNIVERSITY

A recursive solution has three obligations

Before the first call

1. State the valid input domain.
2. Identify all smallest cases answered directly.
3. Show that every recursive call reduces a well-founded problem size.

After the smaller call

1. Receive its returned result.
2. Combine it with the current state.
3. Return the current answer to the waiting caller.

A base case is not enough

If positive n is replaced by $n + 1$, a base case at zero exists but is unreachable. A recursion-depth failure is not a successful result.

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Factorial: descend, then unwind

```
def factorial(n):
    """Return n! for a nonnegative built-in
     integer; use small n."""
    if type(n) is not int:
        raise TypeError("n must be an
     integer, not bool")
    if n < 0:
        raise ValueError("n must be
     nonnegative")
    if n == 0:
        return 1  # Base case: the neutral
     value for multiplication.
    return n * factorial(n - 1)
```

Trace factorial(4)

$4 \cdot \text{fact}(3)$
$4 \cdot 3 \cdot \text{fact}(2)$
$4 \cdot 3 \cdot 2 \cdot \text{fact}(1)$
Base: $\text{fact}(0) = 1$.
Returned values: 1, 1, 2, 6, 24.

[W06] factorial; this checked review version explicitly excludes bool and raises exceptions using Week 7 ideas.

Contents

Fibonacci uses the established course convention

```
# Course convention: F(0) = F(1) = 1; small
    n only.
def fibonacci(n):
    """Caller supplies an integer n >= 0."""
    if n == 0 or n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n -
    2)
```

Two base cases, two recursive calls

$F(2) = 2$, $F(3) = 3$, $F(4) = 5$, $F(5) = 8$. Both inputs decrease, but the same subproblem is recomputed in different branches.

Do not silently switch conventions

Some references start Fibonacci with 0 and 1. These course examples use 1 and 1. A correct implementation must satisfy the specified convention.

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Iteration can retain only the required recent state

```
# Caller supplies a nonnegative integer n.
def fibonacci_iterative(n):
    previous, current = 1, 1
    for index in range(2, n + 1):
        # Evaluate both right-hand
     expressions first.
        previous, current = current,
     previous + current
    return current
```

Trace n = 5

Start (1,1). After indices 2, 3, 4, 5: (1,2), (2,3), (3,5), (5,8). The loop does not recompute an earlier term.

Resource distinction

Recursive work, maximum depth, and the size of stored numeric values are different resource questions. Weeks 10–11 analyze these explicitly.

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A non-numeric problem can also become smaller

```
def is_palindrome(text):
    """Test the exact, case-sensitive
     characters of a string."""
    if len(text) <= 1:
        return True  # Empty and one-
     character strings are base cases.
    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))
```

Trace the exact string

"level" compares l with l, then "eve", then the one-character base case "v".
A mismatch short-circuits the and: no further recursive call is needed.

Specify normalization separately

This function is case-sensitive and includes spaces and punctuation. Cleaning before calling it would be a separate, documented task.

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Dictionaries retrieve by key, not position

Operation	Meaning
`record["value"]`	Retrieve a value; absent key raises KeyError.
`record.get("value")`	Retrieve a value or the default None.
`"value" in record`	Test membership of keys, not values.
`record["value"] = 25`	Insert or update the named entry.
`record.items()`	Iterate through key-value pairs.
`record.pop("value")`	Remove the entry and return its value.

A schema makes a record interpretable

Agree on key spelling, units, expected types, and missing-value behavior. Keys must be hashable; use simple strings for these engineering records.

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Nested records keep properties together

```
# Values are illustrative teaching
    properties, not specifications.
materials = {
    "steel": {"density_kg_m3": 7850},
    "aluminum": {"density_kg_m3": 2700}
}
selected = "steel"
density = materials[selected]["density_kg_m3
    "]
print(density)
print(materials.get("titanium"))
```

Two lookups, two possible failures

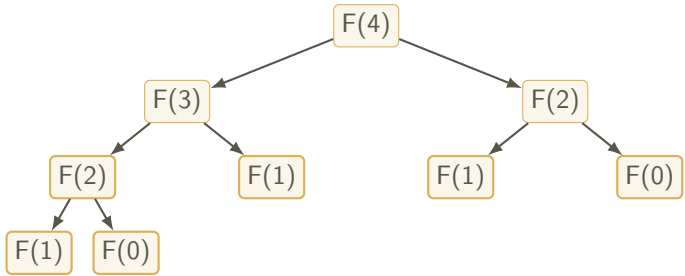
The outer key selects the material. The inner key selects the property. Knowing that a material exists does not prove that every desired property exists.

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A complete small Fibonacci call tree



Count the actual nodes

There are nine calls in this complete tree, including repeated base cases. F(4) returns 5 under the course convention. The two F(2) calls perform the same subproblem separately.

Contents

[W06] multiple-base-case recursion; complete tree clarifies the missing leaf in the earlier drawing. Executed call-count check supplied.

NAZARBAYEV UNIVERSITY

M06-Q1 | Before or after the recursive call?

```
# Observe the return path rather than only
    call order.
def show(n):
    if n == 0:
        return
    show(n - 1)
    print(n)
show(3)
```

Write your prediction first

1. Predict the output order.
2. What becomes smaller?
3. How would moving print above the recursive call change the result?

▸ Worked answer

Contents

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

M06-Q1 | Worked answer

```
1
2
3
```

Why this happens

The base case returns first. Waiting calls then print 1, 2, 3 during unwinding. Moving print above the call prints 3, 2, 1 during descent. The nonnegative integer n decreases by one.

Return to question

Contents

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

NAZARBAYEV UNIVERSITY

M06-Q2 | Choose a missing-value policy

```
# A zero reading and an absent record are
    different facts.
readings = {"P-01": 0.0, "P-02": 120.0}
print("P-01" in readings, 120.0 in readings)
print(readings.get("P-01"))
print(readings.get("P-03"))
print("P-03" in readings)
```

Write your prediction first

1. Predict every printed line.
2. Why is membership different for a key and value?
3. Does get insert a missing entry?

▸ Worked answer

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

M06-Q2 | Worked answer

```
True False
0.0
None
False
```

Why this happens

Membership tests keys. The numeric zero is a present, valid value, whereas the missing key gives `None`. Calling get does not insert the key.

Return to question

Contents

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

NAZARBAYEV UNIVERSITY

M06-Q3 | A recurrence is not a loop

Try before viewing the solution

For $R(0) = 0$ and $R(n) = n + R(n-1)$ for positive integer n, expand R(3), state its meaning, and compare a recursive implementation with a running total.

▸ Worked answer

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M06-Q3 | Worked answer

Worked reasoning

$R(3) = 3 + (2 + (1 + 0)) = 6$. It sums integers 1 through n. Both straightforward versions perform n additions under a unit-cost arithmetic model. Recursion retains a chain of waiting calls; an iterative running total does not. The stated domain must exclude negative inputs.

Return to question

[W06] recursion, multiple base cases, strings, lists, dictionary keys and values. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

07 | Testing, debugging, and failure

Week 7: require evidence, not merely successful execution

NAZARBAYEV UNIVERSITY

Testing and debugging answer different questions

Testing

Run selected inputs and compare actual behavior with an independently justified expectation. A failing test supplies a reproducible symptom.

Debugging

Investigate the symptom, locate the cause, make a controlled repair, and retain a regression test.

Correct with respect to what?

State types, units, valid ranges, output accuracy, mutation behavior, and expected failures. Passing tests support the stated contract; they do not prove it for every input or validate the physical model.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Choose cases for different reasons

Test choice	What it examines	Example
Known-answer	Independent expected value.	10 N / 2 m^2 = 5 Pa.
Typical	Ordinary intended use.	A representative measured value.
Boundary	Equality and neighboring regions.	19.9, 20, 30, 30.1 °C.
Invalid input	Documented rejection.	Zero area or malformed text.
Non-mutation	Preservation of caller data.	Raw readings equal their saved copy.
Regression	A repaired defect stays fixed.	Keep the exact FoS = 2 test.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Contract tests and path tests complement each other

Black-box

Use the specification without examining the body. Partition valid and invalid input classes and test important boundaries.

Glass-box

Inspect the body. Exercise alternative branches, zero/one/several loop passes, and recursive base and return routes.

Coverage is not correctness

A function can execute every branch while all its returned values are wrong. A missing requirement can be absent from the implementation entirely.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Error categories guide the first investigation

Observed category	First useful question
Syntax error	Are quotes, colons, parentheses, and indentation valid?
Runtime exception	Which operation failed and what values reached it?
Logic error	Where does the state first disagree with the expected trace?
Unit or data error	Are units, field meanings, and record associations correct?
Numerical failure	Did the method satisfy its accuracy and stopping requirement?

Read a traceback

Start with the exception type and message, then the failing line, then the caller that supplied the values. The detected location may not be the original cause.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Debugging is a controlled experiment

1. Reproduce the failure with the same source and inputs.
2. Reduce the example while preserving the failure.
3. Print selected values, types, units, and repr(text) where needed.
4. Predict how one proposed repair should change the result.
5. Repair that cause; rerun the failed test and the earlier suite.

Keep the evidence

Record input, expected output, observation, cause, repair, and regression case. Random edits make it difficult to know what actually fixed the problem.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Use exceptions to report an invalid operation

```
# A minimal domain check; finite numeric
    input is assumed.
def stress_pa(force_n, area_m2):
    if area_m2 <= 0:
        raise ValueError("area must be
    positive")
    return force_n / area_m2
try:
    result = stress_pa(10.0, 0.0)
except ValueError as error:
    print("Rejected:", error)
else:
    print("Stress:", result, "Pa")
```

Trace only the path taken

A raised exception skips the remaining statements in the try block. A matching handler runs. The else block runs only after a successful try block.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Assertions are checks, not permanent input barriers

```
# Executable expectations for selected tests
    .
from review_models import stress_mpa
assert stress_mpa(10_000, 500) == 20.0
assert stress_mpa(0, 500) == 0.0
# Approximate comparisons need a justified
    criterion.
assert abs((0.1 + 0.2) - 0.3) < 1e-12
```

Choose the mechanism

Use explicit validation and meaningful exceptions for invalid external input. Use assertions to express expectations and internal consistency during development.

Important limitation

Assertions can be disabled by optimized Python execution. They must not be the sole protection for required user-input checks.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A complete test checks state as well as output

Functions

Check returned values, documented exceptions, input preservation, and correctness of the composed pipeline.

Objects

Check initial state, valid updates, rejected updates, and independence of separate instances. A failed update should preserve the previous state when promised.

Trend checks support, not replace, known answers

For $\sigma = F/A$, doubling force doubles stress; doubling area halves it. A wrong constant unit multiplier can still pass both trend checks.

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M07-Q1 | Two unit defects survive execution

Try before viewing the solution

A script uses `area_m2 = 500 / 1000`, then divides 10,000 N by that area, then divides strength 250 MPa by that stress in Pa. Find both errors and give the correct stress and FoS.

▸ Worked answer

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M07-Q1 | Worked answer

Worked reasoning

Area requires 10^{-6}, not 10^{-3}: 500 mm^2 = 0.0005 m^2. Stress is 20,000,000 Pa = 20 MPa. Strength and stress must use the same unit, giving FoS = 250/20 = 12.5. The defects concern meaning, not Python syntax.

Return to question

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M07-Q2 | Trace exception control flow

```
# A handled conversion failure does not
    produce a fake value.
try:
    value = float("bad")
    print("converted")
except ValueError:
    print("rejected")
else:
    print("success")
print("finished")
```

Write your prediction first

1. Write the printed lines in order.
2. Does the second print in try run?
3. Why is else skipped?

▸ Worked answer

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

M07-Q2 | Worked answer

```
rejected
finished
```

Why this happens

Converting "bad" raises ValueError, so the next statement in try is skipped. The matching handler prints "rejected". The successful-only else block is skipped. Execution continues afterward.

Return to question

Contents

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

NAZARBAYEV UNIVERSITY

M07-Q3 | Design the tests before the function

Try before viewing the solution

Contract: a function returns the mean of a nonempty finite numeric list, rejects empty input, and does not change the list. Propose four known-answer cases, an invalid case, and a state check.

▸ Worked answer

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M07-Q3 | Worked answer

Worked reasoning

Examples: [2,4] gives 3; [5] gives 5; [3,3,3] gives 3; [-2,2] gives 0. Empty input must trigger the documented rejection. Save a copy before the call and assert equality afterward. Include nonfinite values if the implementation promises to reject them.

Return to question

Contents

[W07] test classes, black-box/glass-box tests, debugging, exceptions, assertions. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

08 | Classes, instances, and methods

Week 8: keep related state and behavior together

NAZARBAYEV UNIVERSITY

OOP is an organizational choice

Term	Meaning	Sensor example
Class	A definition of a type and its operations.	`IntroSensor`
Instance	One particular object of that type.	`temperature`
Data attribute	Named information associated with an instance.	`temperature.value`
Method	An operation called with a receiver.	`temperature.report()`
State	Current stored information.	Current reading and limits.

Choose by responsibility

Functions plus dictionaries may be clearer for a small task. A class helps when a meaningful entity has related data and operations.

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Initialize the expected attributes together

```
# Separate name prevents confusion with later checked models.
class IntroSensor:
    def __init__(self, name, unit, value, lower, upper):
        # self is this particular new instance.
        self.name = name
        self.unit = unit
        self.value = value
        self.lower = lower
        self.upper = upper
```

Local input becomes stored state

In `self.value = value`, the right side is a local argument; the left side stores it on the instance. The initializer establishes state and must not return a non-None result.

Contents

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

NAZARBAYEV UNIVERSITY

An ordinary method receives the selected instance

```
# Method excerpt inside IntroSensor; full
    file is supplied.
def report(self):
    """Return text; do not change the stored
     reading."""
    return f"{self.name}: {self.value:.1f} {
    self.unit}"
```

Binding

For this ordinary method, `s.report()` supplies `s` as the first argument. It has the same effect here as `IntroSensor.report(s)`.

Read versus call

`s.value` retrieves a value. `s.report` retrieves a method object. `s.report()` calls it. Calling `s.value()` is an error when the attribute is a number.

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Query and update are separate responsibilities

```
# Excerpts: the caller supplies ordered limits and finite values.
def is_safe(self):
    """Mean only: within the stated inclusive limits."""
    return self.lower <= self.value <= self.upper
def update(self, new_value):
    # This simple version assumes the input is valid.
    self.value = new_value
    # No explicit return: the result of the call is None.
```

Two separate effects

The query returns a Boolean without changing the reading. The update changes state and returns None. Here is-safe means only compliance with the stated interval, not physical certification.

Contents

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

NAZARBAYEV UNIVERSITY

Keep instance identity and copied values distinct

Alias

`b = a` gives another name to the same object. Updating through `b` is visible through `a`.

Saved scalar

`saved = a.value` stores the current scalar. Rebinding `saved` later does not update `a`; updating `a` later does not recompute `saved`.

Tests for two instances

Construct separate objects, update one, and check that the other remains unchanged. Distinct instances should not accidentally share mutable storage unless the contract calls for it.

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M08-Q1 | Saved query or live condition?

```
# The complete class is in examples/
    intro_sensor.py.
from intro_sensor import IntroSensor
s = IntroSensor("Tank", "deg C", 25.0, 20.0,
     30.0)
old_ok = s.is_safe()
result = s.update(35.0)
print(old_ok, s.is_safe(), result)
```

Write your prediction first

1. Predict the three results.
2. Why does the saved Boolean not change?
3. Does the method update return the sensor?

▸ Worked answer

Contents

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

M08-Q1 | Worked answer

```
True False None
```

Why this happens

The saved Boolean reflects the earlier reading of 25.0. After updating to 35.0 the current query is False, but the saved Boolean remains True. The update mutates the object and returns None.

Return to question

Contents

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

NAZARBAYEV UNIVERSITY

M08-Q2 | Missing parentheses change the question

Try before viewing the solution

A program writes `if s.is_safe:  print("normal")`. Explain why the method body is not being queried. Then explain why `s = s.update(35)` loses the reference.

▸ Worked answer

Contents

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

NAZARBAYEV UNIVERSITY

M08-Q2 | Worked answer

Worked reasoning

Without parentheses, the is-safe attribute refers to the bound method object, which is truthy; it does not evaluate the limit condition. Use `s.is_safe()`. The update method returns None, so assigning its result back to s replaces the reference with None. Call `s.update(35)` on its own.

Return to question

Contents

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

NAZARBAYEV UNIVERSITY

Week 8 checkpoint

- Explain the class, instance, local parameter, and attribute in one example.
- Trace which object receives a method call.
- Track state changes separately from returned results.
- Test initialization, exact boundaries, and independent instances.

Learning boundary

The simple IntroSensor assumes valid input. The next module reviews explicit validation and deliberate public interfaces rather than pretending these guarantees already exist.

[W08] OOP advantages, attributes, methods, instance creation, state tests. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

09 | Interfaces, shared state, inheritance

Week 9: control changes and reuse behavior deliberately

NAZARBAYEV UNIVERSITY

A setter should validate before assignment

```
# Methods from the checked review Sensor
    class.
def get_value(self):
    return self._value
def set_value(self, proposed):
    # finite_number rejects invalid types
    and nonfinite values.
    checked = finite_number(proposed, "
    reading")
    self._value = checked
```

Atomic update promise

Either the proposal is accepted and stored, or it is rejected before replacing the old reading. A valid reading outside operating limits should be retained and flagged, not discarded.

Different types of invalidity

A negative absolute pressure can be invalid for that domain. A negative Celsius temperature is not automatically invalid. Validation must match the quantity.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Information hiding is about the interface

Public promise

The caller should know the supported methods, units, constraints, and returned results. Internal representation can change without changing the promise.

Python naming conventions

A leading underscore signals non-public implementation detail. It is not access control or a security boundary.

Mutable-return caution

A getter returning its internal list gives the caller a path to mutate it. A copied outer list protects outer changes only; nested mutable objects may remain shared.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Class defaults and instance overrides

```
# Reporting preference shared by default,
    not one shared reading.
class Display:
    digits = 1
a = Display()
b = Display()
a.digits = 3  # Create an instance override
    on a.
Display.digits = 2
# a uses its own 3; b still uses the class
    default 2.
```

Lookup in this simple case

An instance attribute is used when present. Otherwise Python can find the class attribute. Reassigning the class default does not erase an instance override.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A class-level mutable list can couple instances

```
# Intentional design defect for independent
    histories.
class BadLog:
    history = []
    def add(self, value):
        self.history.append(value)
# Both instances find the same shared class
    list.
```

```
# Repair: create storage during each
    initialization.
class GoodLog:
    def __init__(self):
        self.history = []
    def add(self, value):
        self.history.append(value)
```

Regression test

Create two logs; append through one; verify that the other's history remains empty.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Inherit a meaningful interface

```
class TemperatureSensor(Sensor):
    """Specialize the unit and report; retain the parent's update rule."""
    def __init__(self, name, value, lower=20.0, upper=30.0):
        super().__init__(name, "deg C", value, lower, upper)
    def report(self):
        # Reuse the parent report and add a type-specific prefix.
        return "Temperature | " + super().report()
```

Preserve the public promise

The child reuses initialization and validation and adds a report prefix. Inherited methods still act on this particular child instance.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Inheritance and composition express different relationships

Is a

A TemperatureSensor is a Sensor specialization. Shared interface and behavior can be inherited.

Has a

An experiment has sensors. Store them as attributes or a collection rather than making an experiment inherit from Sensor.

Do not build hierarchy for its own sake

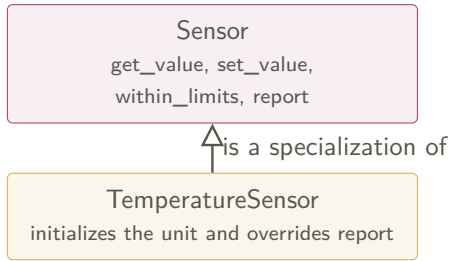
Prefer the simplest structure that makes responsibilities, state, and tests clear. A class name alone does not establish a sound relationship.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Follow inherited calls and their contracts



Receiver identity stays important

An inherited method still acts on the particular child instance. A parent method can call a method selected through that receiver. Test the externally observable contract after overriding behavior.

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M09-Q1 | Which precision value is found?

```
# Trace attribute lookup for one shared
    default.
class Display:
    digits = 1
a = Display()
b = Display()
a.digits = 3
Display.digits = 2
print(a.digits, b.digits, Display.digits)
```

Write your prediction first

1. Predict the three integers.
2. Which object has an override?
3. Why does changing the class affect b?

▸ Worked answer

Contents

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

M09-Q1 | Worked answer

```
3 2 2
```

Why this happens

a has its own digits = 3. b has no override and therefore uses the changed class default 2. The class itself stores 2.

Return to question

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M09-Q2 | Does rejection preserve state?

```
# The checked review model validates before
    storing.
from review_models import TemperatureSensor
sensor = TemperatureSensor("Tank", 25.0)
try:
    sensor.set_value(float("nan"))
except ValueError:
    print("rejected")
print(sensor.get_value())
```

Write your prediction first

1. Predict both printed lines.
2. Which statement prevents overwriting the old value?
3. Is 35 °C a valid but out-of-limit proposal for this model?

▸ Worked answer

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

M09-Q2 | Worked answer

```
rejected
25.0
```

Why this happens

The finite-number helper rejects NaN before assignment, so the stored reading remains 25.0. A finite proposal of 35.0 would be stored, while the limit query would return False for the 20–30 interval.

Return to question

Contents

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

NAZARBAYEV UNIVERSITY

M09-Q3 | Parent initialization was skipped

Try before viewing the solution

A child defines its own initializer but never calls the parent initializer that creates name, limits, and value. Its inherited report then reads those attributes. What failure is likely, and what should be tested after repair?

▸ Worked answer

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M09-Q3 | Worked answer

Worked reasoning

An AttributeError is likely when an expected attribute is missing. Call the appropriate parent initializer with the required arguments (or deliberately establish the same contract). Test initial state, inherited query boundaries, a valid update, a rejected update, and independent instances.

Return to question

Contents

[W09] getters, setters, information hiding, class variables, inheritance. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

10 | Efficiency: counts and growth

Week 10: state the cost model before the complexity class

NAZARBAYEV UNIVERSITY

A complexity claim needs a complete sentence

1. Define the input size: n readings, n candidates, or another measure.
2. Specify the input case: best, worst, or an average under stated assumptions.
3. Choose the operation counted: comparisons, arithmetic updates, copies, or total work.
4. State a model: bounded-cost arithmetic, comparisons, and indexing when assumed.

Example

A full scan of n readings performs n threshold comparisons. Under bounded-cost operations, its time is $\Theta(n)$. That is not a measurement of n seconds.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Count comparisons, not just successful events

```
# Finite readings; this routine must inspect
      every reading.
def count_warnings(readings, limit):
    count = 0
    for value in readings:
        if value > limit:
            count += 1
    return count
```

Two different counts

Every input contributes one comparison. Only readings above the limit contribute an increment. For n inputs, the number of warnings can be 0 while the number of comparisons is n.

Change the output requirement, change the work

Finding only the first warning may stop early. Counting all warnings generally cannot stop after finding the first.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Big-O is an asymptotic upper bound

Definition

$T(n) = O(g(n))$ if positive constants c, n_0 exist such that $0 \le T(n) \le c\,g(n)$ for all $n \ge n_0$.

Worked bound

$3n + 20 \le 4n$ when $n \ge 20$. Thus $3n + 20 = O(n)$.

Tight growth

$\Theta(g(n))$ indicates matching asymptotic upper and lower bounds. $3n + 20 = \Theta(n)$.

Keep case and notation separate

Big-O does not itself mean worst case. First define the cost function and the input case; then give its bound.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Dominant terms describe growth, not small-input winners

As input grows

$1000 + n + 2n^2 = \Theta(n^2)$ under the stated count model. Constants and lower-order terms do not change its asymptotic class.

At one finite size

$100n$ is larger than n^2 when $0 < n < 100$, equal at 100, and smaller afterward. Asymptotic class does not settle every small case.

No time units are implied

These expressions are declared operation models. Do not relabel them as measured milliseconds.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Cases depend on how the input is arranged

First-match linear search	Comparisons	Interpretation
Target at first position	1	Best case for a nonempty list.
Target at last position	n	A worst case.
Target absent	n	Another worst case.
Uniformly likely successful position	$(n+1)/2$	Average only under this stated model.

Do not confuse workload questions

Returning all matches requires inspecting later candidates even when the first item matches.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Recognize the major growth classes

Class	Characteristic pattern	Doubling input size
$O(1)$	Fixed number of existing-list accesses.	Constant work.
$O(\log n)$	Repeatedly halve a candidate interval.	One additional halving level.
$O(n)$	One full scan.	About twice the work.
$O(n \log n)$	Linear work per halving level.	A little more than twice.
$O(n^2)$	All pairs of positions.	About four times.
$O(n^3)$	Three independent size-n loops.	About eight times.
$O(2^n)$	Exponential branching or binary choices.	Growth is far more severe.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Combine costs from structure, not indentation alone

Add sequential work

One n-scan followed by another n-scan gives $2n = \Theta(n)$.
A scan followed by an n^2 pair loop gives $\Theta(n^2)$.

Multiply independent repetitions

n outer passes, each doing n work, give $\Theta(n^2)$.
n passes, each doing 5 work, give $\Theta(n)$.

Dependent bounds need a sum

A triangular loop gives $1 + 2 + \cdots + (n-1) = n(n-1)/2$, not n^2 exact visits; both have quadratic growth.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Measure an implementation fairly

```
# Keep data construction and printing
    outside the timed section.
from time import perf_counter
values = list(range(10_000))
start = perf_counter()
total = sum(values)
elapsed_s = perf_counter() - start
assert total == 49_995_000  # Check after
    timing.
print(elapsed_s)
```

Repeat and document

- Use the same input and output task.
- Repeat the measurement.
- Record machine and runtime information.
- Check correctness independently.
- Do not report an invented universal speed.

Example status

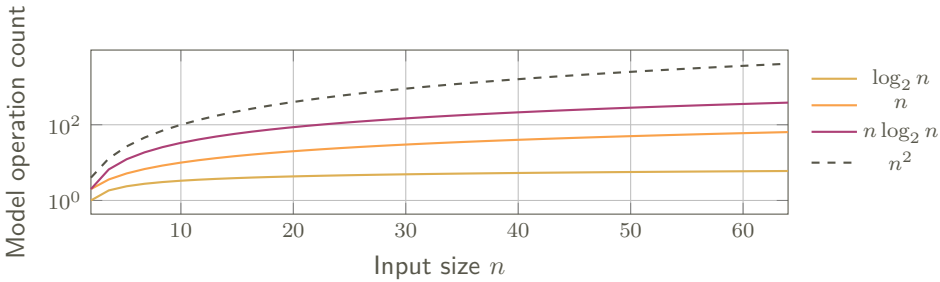
This is a benchmarking recipe. No fixed elapsed time is promised or printed in the answer material.

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Growth curves are models, not benchmark results



Read the axes

The vertical axis is logarithmic to keep different growth rates visible. These are declared mathematical count functions, not measurements of a machine.

Contents

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

NAZARBAYEV UNIVERSITY

M10-Q1 | A fixed inner bound

```
# Count selected visits, not elapsed time.
n = 4
visits = 0
for i in range(n):
    for j in range(5):
        visits += 1
print(visits)
```

Write your prediction first

1. Predict the count.
2. Express the count for general nonnegative n.
3. Give its tight growth under a unit-cost model.

▸ Worked answer

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

M10-Q1 | Worked answer

20

Why this happens

The inner loop has exactly five passes independently of n. The count is 5n, so for $n = 4$ it is 20. Its tight growth is $\Theta(n)$, despite being written as two nested loops.

Return to question

Contents

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

NAZARBAYEV UNIVERSITY

M10-Q2 | An upper bound need not be tight

Try before viewing the solution

A comparison count is $T(n) = 3n + 20$. Is $O(n^2)$ a valid upper bound? Is $\Theta(n^2)$ the correct tight growth? Explain both answers.

▸ Worked answer

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M10-Q2 | Worked answer

Worked reasoning

$O(n^2)$ is a valid but loose upper bound for large n. The tight growth is $\Theta(n)$, not $\Theta(n^2)$. Giving the loosest true bound can hide a useful distinction between algorithms; specify a tight bound when you can justify it.

Return to question

[W10] measuring work, orders of growth, Big-O, complexity classes. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

11 | Hidden work, recursion, and memory

Week 11: classify the complete implementation

NAZARBAYEV UNIVERSITY

One line can hide substantial work

Operation	Cost model	Qualification
`values[i]`	$O(1)$	Existing list; valid index.
`sum(values)`	$O(n)$	Bounded-cost additions.
`values[a:b]`	$O(k)$	Copies k references.
`values.pop(0)`	$O(n)$	Shifts later references.
`values.append(x)`	$O(1)$ amortized	A resize can cost more.
`key in mapping`	$O(1)$ expected	Not a worst-case guarantee.

State the model

Hashing, comparisons, arithmetic size, and required output can change the full cost. These are the collection assumptions used in Weeks 10–11.

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Repeated copying versus incremental construction

```
# Deliberately inefficient: copy the growing
      prefix repeatedly.
result = []
for value in values:
    result = result + [2 * value]
```

```
# Same output order and values; reuse one
    result list.
result = []
for value in values:
    result.append(2 * value)
```

Explain the difference

The first version repeatedly copies about $0 + 1 + \cdots + (n - 1)$ existing references: quadratic total copying. Append supports linear total construction under its amortized model. Both still store n output values.

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Preprocessing can change repeated-query cost

Repeated list search

For n installed IDs and m approved IDs, n list-membership checks can cost $O(nm)$ in the worst case.

Build one dictionary index

Build an index of approved IDs once; then check each installed ID. Expected total work is $O(m + n)$ for short hashable IDs under the usual lookup model.

Keep the contract

Return one result per installed ID, preserving order and duplicates. Include index construction, memory, and update costs rather than comparing only a prebuilt lookup with a full scan.

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Recursive depth and total work are not the same

Recursive pattern	Work under stated assumptions	Live recursive depth
One call on n-1; constant local work	$\Theta(n)$	$\Theta(n)$
One call on n/2; constant local work	$\Theta(\log n)$	$\Theta(\log n)$
Two disjoint half calls; linear merge	$\Theta(n \log n)$	$\Theta(\log n)$
Naive overlapping Fibonacci	Exponential total calls.	$\Theta(n)$

Do not count only the longest path

A call tree may contain many branches even when each path is short. Conversely, a long chain can do linear work.

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

Passing an index avoids repeated tail copies

```
# Same list, changing index; short inputs
    for recursion practice.
def sum_index(values, index=0):
    if index == len(values):
        return 0
    return values[index] + sum_index(values,
     index + 1)
```

```
# Each recursive call creates a new tail
    list.
def sum_slice(values):
    if not values:
        return 0
    return values[0] + sum_slice(values
    [1:])
```

Under bounded-cost arithmetic

Index recursion has linear work and linear frame depth. Tail slicing adds quadratic copying; retained tails can also require quadratic live reference storage. A simple iterative sum avoids both the call chain and the copies.

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Caching changes overlapping work, not the recurrence

```
# Excerpt: n is a valid nonnegative integer.
# A fresh cache is initially {0: 1, 1: 1}.
def cached(n, cache):
    if n not in cache:
        cache[n] = (cached(n - 1, cache)
                    + cached(n - 2, cache))
    return cache[n]
```

Fresh computation versus later lookup

With a fresh cache, each new sequence value is computed once: linear distinct states under unit-cost arithmetic. An already cached lookup answers a different workload question.

Memory and numeric scale

The cache stores O(n) values and recursion can have O(n) depth. Python's growing integers do not have constant bit size; the simple count model abstracts from that cost.

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Moving-window means: specify all outputs

Contract

For n readings and width $1 \leq w \leq n$, return each of the $n - w + 1$ full-window means in source order.

Direct computation

Sum w values for each window. Selected additions: $w(n - w + 1)$. The worst choice of w grows quadratically with n.

Running subtotal

Sum the first w values, then remove one old value and add one new value. Selected updates: $w + 2(n - w) = 2n - w$.

Define what is counted

These counts concern the selected arithmetic updates under the Week 11 convention. Loop tests, output writes, and division are additional costs.

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

Faster algebra does not guarantee required accuracy

```
# A floating-point counterexample for
    repeated subtotal updates.
from review_models import
    window_means_direct
from review_models import
    window_means_running
values = [1e16, 1.0, 1.0]
print(window_means_direct(values, 2))
print(window_means_running(values, 2))
```

Inspect the second window

The exact mean of [1.0, 1.0] is 1.0. The first large subtotal loses the small contribution through rounding; subtracting the large term later cannot recover it.

Decision criterion

An optimization must preserve the required error tolerance, not merely the mathematical formula. Keep a numerical regression test.

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

M11-Q1 | A shrinking inner workload

```
# Inner work shrinks along with the outer
    state.
size = 8
visits = 0
while size > 0:
    for _ in range(size):
        visits += 1
    size //= 2
print(visits)
```

Write your prediction first

1. Predict the selected visit count.
2. Write the sum of inner-loop lengths.
3. Give tight asymptotic growth for positive n.

▸ Worked answer

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

M11-Q1 | Worked answer

15

Why this happens

The lengths are 8, 4, 2, 1, giving 15. In general $n + \lfloor n/2 \rfloor + \cdots < 2n$, while the first stage already takes n visits. Thus tight growth is $\Theta(n)$, not quadratic.

Return to question

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

M11-Q2 | A helper call hides a full scan

Try before viewing the solution

A program subtracts the mean from each reading, but recomputes `sum(readings)/len(readings)` inside the output loop. Classify its cost and propose a contract-preserving repair.

▸ Worked answer

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M11-Q2 | Worked answer

Worked reasoning

For n unchanged readings it performs n full sums, giving quadratic work under bounded-cost arithmetic. Compute the mean once before the loop, then build the centered output in a single scan. Handle empty input as the contract specifies and preserve the raw list.

Return to question

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

M11-Q3 | Operation count and output count differ

Try before viewing the solution

For n = 6 and width w = 3, how many full-window means are required? Compute the selected arithmetic counts for direct and running-subtotal versions. Does fewer updates alone establish the better implementation?

▸ Worked answer

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

M11-Q3 | Worked answer

Worked reasoning

There are $6 - 3 + 1 = 4$ means. Direct count: $3 \times 4 = 12$. Running count: $2 \times 6 - 3 = 9$. Fewer selected updates do not alone establish lower measured runtime or adequate accuracy. Compare the same outputs, numerical tolerance, memory, and realistic execution conditions.

Return to question

Contents

[W11] hidden costs, recursion, memory, moving-window computations, scaling. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

12 | Searching and sorting

Week 12: preserve the question, the records, and the order

NAZARBAYEV UNIVERSITY

Specify the search result

Question	Return contract	Can stop at first success?
Is the target present?	Boolean.	Yes.
Where is its first occurrence?	First index or None.	Yes, in source-order scan.
Which records qualify?	All qualifying records.	No.
Which feasible record is cheapest?	Best record under a tie policy.	Not generally.
Where could a target be inserted?	Insertion position.	Not an existence claim.

State absence and tie policies

Zero is a valid index. None or another explicit marker should indicate absence under the stated interface.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Linear search: absence is decided after the loop

```
def linear_search(values, target):
    """Return the first matching index, or
     None; do not reorder values."""
    for index, value in enumerate(values):
        if value == target:
            return index  # Zero is a valid
     successful result.
    return None  # Absence is reported only
     after all candidates fail.
```

Correctness reasoning

Before each pass, all earlier positions have been tested without a match. Returning inside the match branch gives the first occurrence. Returning None belongs after all candidates fail.

Tests

Empty input; singleton match and nonmatch; first and last positions; duplicates; absent target. No sortedness requirement is needed.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Brute-force pair search

```
def feasible_pairs(resistors, target):
    """List unordered index pairs whose series resistance equals target."""
    pairs = []
    for i in range(len(resistors)):
        for j in range(i + 1, len(resistors)):
            if resistors[i] + resistors[j] == target:
                pairs.append((i, j))  # Do not reuse the same component.
    return pairs
```

Contract and candidate set

Return all unordered pairs of different positions with the required series resistance. Starting j at i + 1 avoids reusing a component or testing reversed duplicates.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Binary search needs ordered data and progress

```
def binary_search(values, target):
    """Ascending input; return any match index or None."""
    low, high = 0, len(values) - 1
    while low <= high:
        mid = (low + high) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            low = mid + 1  # Make progress.
        else:
            high = mid - 1
    return None
```

Precondition and invariant

Values are ascending and support efficient indexed access. All remaining possible matches are between low and high, inclusive. Each failed midpoint test removes that midpoint and one side.

Contents

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

NAZARBAYEV UNIVERSITY

Trace a successful binary search

low	high	mid	value[mid]	Target 31: next action
0	6	3	22	Set low = 4.
4	6	5	40	Set high = 4.
4	4	4	31	Return index 4.

Data

```
[8, 12, 18, 22, 31, 40, 55]
```

Important boundary

When low equals high, one candidate remains and must be tested. When low exceeds high, the candidate interval is empty and absence can be reported.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Insertion positions are not equality results

```
# Input is sorted; bisect_left returns an
    insertion index.
from bisect import bisect_left
values = [10, 20, 20, 40]
target = 30
pos = bisect_left(values, target)
found = pos < len(values) and values[pos] ==
     target
# pos is 3 but found is False.
```

What leftmost means

For duplicate 20s, the left-insertion operation gives the first permitted insertion position, 1. For an absent target such as 30 it still returns a position. Check bounds before reading the item.

Preserve original locations

An index in a sorted copy is not generally the original measurement index. Retain record IDs or original indices when sorting.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Built-in sorting: mutation and keys

```
# Preserve raw order with sorted; sort
    mutates in place.
raw = [3, 1, 2, 1]
ranked = sorted(raw)
result = raw.sort()
# ranked and raw are [1, 1, 2, 3]; result is
    None.
```

Sorting obligations

- Values are in the requested order.
- No item or duplicate disappears.
- Fields of each record stay together.
- Mutation matches the promise.
- Ties follow the stated policy.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A key function ranks the entire record

```
# Keep name, density, and every other field
    together.
records = [("steel", 7850), ("aluminum",
    2700)]
def density_key(record):
    return record[1]
ranked = sorted(records, key=density_key)
print(ranked)
print(records[0][0])
```

Stability

A stable sort preserves the original relative order of records with equal keys. Add a second key only when the requirement specifies a tie-breaker.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Selection sort fixes one position each pass

```
def selection_sort(values):
    """Return a sorted copy using selection sort; assume ordered values."""
    result = values.copy()  # Preserve the input sequence.
    for start in range(len(result) - 1):
        smallest = start
        for index in range(start + 1, len(result)):
            if result[index] < result[smallest]:
                smallest = index
        result[start], result[smallest] = result[smallest], result[start]
    return result
```

Work and mutation

The standard full-suffix search makes $n(n-1)/2$ comparisons. This implementation returns a new outer list. Swap-based selection sort is not generally stable for equal-key records.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Merge sort: divide, solve, and combine

```
# Structure: merge() must stably merge two sorted lists.
def merge_sort(values):
    if len(values) <= 1:
        return values.copy()
    mid = len(values) // 2
    left = merge_sort(values[:mid])
    right = merge_sort(values[mid:])
    return merge(left, right)
```

Reasoning

Each half is a smaller problem. The merge stage must keep every element. Balanced levels and linear merging give O(n log n) worst-case work under bounded-cost comparison. The full executable implementation is supplied.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Merge without losing items

```
# Excerpt: left and right are already sorted lists.
merged, i, j = [], 0, 0
while i < len(left) and j < len(right):
    if left[i] <= right[j]:  # Take left on a tie.
        merged.append(left[i])
        i += 1
    else:
        merged.append(right[j])
        j += 1
merged.extend(left[i:])  # One remainder may be empty.
merged.extend(right[j:])
```

Stability and progress

At least one index advances on each pass. Choosing the left record on equal keys preserves earlier order across the two halves. Appending leftovers prevents lost items.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A method-selection table needs qualifications

Method	Work	Key condition
Linear search	$O(n)$ worst case	No sorting required.
Indexed binary search	$O(\log n)$ worst case	Ascending random-access sequence.
All unordered pairs	$\Theta(n^2)$ candidates	Every pair is requested.
Selection sort	$\Theta(n^2)$ comparisons	Standard full-suffix implementation.
Balanced merge sort	$O(n \log n)$ worst case	Linear merge; bounded key cost.

Include preprocessing and memory

Sorting merely to do one lookup can cost more than one linear scan. Many later queries can justify the preparation. A sliced or copied implementation also uses storage.

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M12-Q1 | Index zero is a successful answer

```
# Preserve the first-match contract and
    explicit absence marker.
from review_models import linear_search
index = linear_search(["A", "B", "A"], "A")
print(index, bool(index), index is not None)
print(linear_search(["A", "B"], "C"))
```

Write your prediction first

1. Predict all results.
2. Why is testing only if index misleading?
3. Which occurrence does the linear search return?

▸ Worked answer

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

M12-Q1 | Worked answer

```
0 False True
None
```

Why this happens

The first A is at index 0, which is false in a truth-value test but is a valid successful result. Use `index is not None`. The absent C gives None.

Return to question

Contents

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

NAZARBAYEV UNIVERSITY

M12-Q2 | Trace a stable sort by one key

```
# Rank complete records by one declared key.
records = [("A", 2), ("B", 1), ("C", 1)]
def key(record):
    return record[1]
print(sorted(records, key=key))
print(records)
```

Write your prediction first

1. Predict both output lines.
2. Why does B remain before C?
3. Did the original list change?

▸ Worked answer

Contents

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

M12-Q2 | Worked answer

```
[('B', 1), ('C', 1), ('A', 2)]
[('A', 2), ('B', 1), ('C', 1)]
```

Why this happens

The key uses only the numeric field. B and C have equal keys, so Python's stable sort preserves their original order. sorted returns a new list; records stays in its original order.

Return to question

Contents

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

NAZARBAYEV UNIVERSITY

M12-Q3 | Trace the merge, including ties

Try before viewing the solution

Merge sorted left = [1, 3, 3] and right = [2, 3, 4], choosing the left item on a tie. Give the output and explain how losing the leftover tail would violate the contract.

▸ Worked answer

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

Contents

M12-Q3 | Worked answer

Worked reasoning

Take left 1, right 2, left 3, left 3, then append right [3,4]. Output: [1,2,3,3,3,4]. Dropping the remaining tail would lose input elements. A sorted prefix alone is not sufficient: preservation of all items and duplicates is also required.

Return to question

Contents

[W12] search contracts, brute force, linear/bisection search, sorting. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

13 | Programming beyond Python

Week 13: translate meaning, contracts, and tests

NAZARBAYEV UNIVERSITY

What transfers, and what must be relearned?

Transferable reasoning

- Inputs, outputs, units, assumptions.
- Algorithms, decomposition, and interfaces.
- Branches, loops, and state changes.
- Tests, failure behavior, and complexity.

Language-specific details

- Syntax, types, and conversion rules.
- Index origins and slice/range conventions.
- Collection operations and copy behavior.
- Library calls and build/run tools.

Course scope

This is a bridge to other tools, not three additional programming courses. The outline's C+ was treated as C++ in Week 13.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

A language is not the same as its editor or library

Layer	Role	Example already encountered
Language	Rules for expressing the program.	Python, C++, Java, MATLAB.
Library	Reusable computational tools.	Python math; numerical and plotting tools.
Editor or IDE	Write, navigate, and debug source.	VS Code or another chosen editor.
Runtime or compiler	Execute or translate the source.	Python interpreter, JVM, C++ compiler.

Do not infer speed from a language label

The algorithm, data structures, libraries, workload, and implementation determine the observed performance. Preserve the task before benchmarking.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Division depends on operand types

Expression / setting	Result	Reason
Python `7 / 2`	3.5	True division.
Python `7 // 2`	3	Floor division.
C++ `double x = 7 / 2;`	3.0 stored	Integer division occurs before assignment.
Java `double x = 7 / 2;`	3.0 stored	Same operand-type issue.
C++/Java `7.0 / 2`	3.5	One operand is floating point.
MATLAB scalar `7 / 2`	3.5	Numeric literals are doubles in this example.

The receiving type does not repair earlier arithmetic

An integer quotient converted afterward to a float still has its fraction discarded.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

MATLAB indexing and operators

Task	Python / NumPy context	MATLAB
First item	`values[0]`	`values(1)`
Last item	`values[-1]`	`values(end)`
Integer sequence 1 to 5	`range(1, 6)`	`1:5`
Elementwise square	NumPy `a**2`	`a.^2`
Matrix multiplication	NumPy `A @ B`	`A * B`
Elementwise multiplication	NumPy `A * B`	`A .* B`

State the object type

Plain Python lists do not provide NumPy-style elementwise arithmetic. MATLAB row and column shapes also affect which operations are valid.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Conditions, loops, and functions retain familiar roles

Python

Indentation defines blocks; colons introduce them. Functions return with `return`. Method calls bind a receiver.

MATLAB, C++, and Java

MATLAB commonly ends blocks with `end`. C++ and Java commonly use braces. Declared types and function/method structure add language-specific rules.

Read from the calculation outward

Locate the model expression first, then trace input binding, the branch or loop, returned results, and error paths. Additional syntax does not change the engineering question.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

Comparison and copying differ

Setting	Relevant distinction
Python strings	== compares contents; `is` compares identity.
Java strings	Use a suitable `equals` content comparison; == compares references.
C++ std::string	== compares string contents.
Python list assignment	Creates another reference to the same list.
Java array assignment	Creates another reference to the same array.
C++ vector value copy	Copying a vector value creates separate vector storage.

Qualification

Element objects or pointers can introduce further sharing. Always test the actual type and copying operation, rather than relying on a language slogan.

Contents

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

NAZARBAYEV UNIVERSITY

Compare code and model separately

Cooling model from Week 13

$T_{i+1} = T_i - \Delta t\, k(T_i - T_{\text{ambient}})$. The same units, initial state, update rule, and sample locations must be preserved.

Implementation agreement

Compare both languages on identical inputs. Check array length, first and last values, and a tolerance for floating-point results.

Mathematical agreement

Where available, compare with $T(t) = T_{\text{ambient}} + (T_0 - T_{\text{ambient}})e^{-kt}$. Agreement between two codes can preserve the same model or discretization error.

Off-by-one check

100 updates with a stored initial state produce 101 samples, not 100.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

AI-assisted translation does not transfer responsibility

1. Provide the existing input/output contract and units.
2. Request the same algorithm before requesting an optimization.
3. Identify language-specific division, indexing, type, and copy rules.
4. Run independent known-answer and failure tests.
5. Record what was changed and reject unsupported claims of correctness.

Practice, not an assessment policy

This is a study workflow. Follow the actual permitted-resource rules for each graded task.

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M13-Q1 | Which value survives assignment?

Try before viewing the solution

In C++ or Java, compare `double a = 7 / 2;` with `double b = 7.0 / 2;`. Then give the corresponding Python expressions that produce a float 3.0 and a float 3.5.

▸ Worked answer

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

M13-Q1 | Worked answer

Worked reasoning

a stores 3.0 because integer division is performed first; b stores 3.5. Python examples are `float(7 // 2)` and `7 / 2`. Converting the already truncated quotient does not restore the lost fraction.

Return to question

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M13-Q2 | An array operation is not a typographic detail

Try before viewing the solution

The intended operation squares each element of a MATLAB numeric row vector. Why is `a.^2` appropriate while `a^2` is a different operation? What must you check when translating from Python?

▸ Worked answer

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M13-Q2 | Worked answer

Worked reasoning

The dot selects elementwise power. Matrix power is a different operation and generally requires a square matrix; a nonsquare row vector cannot be squared as a matrix. Confirm whether the Python object is a NumPy array or a plain list, and check both shape and intended operation.

Return to question

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

NAZARBAYEV UNIVERSITY

M13-Q3 | Build a translation test set

Try before viewing the solution

A translated stress function accepts force in N and positive area in mm^2, returns MPa, and rejects nonpositive area. Give a test set that checks meaning rather than only similar-looking code.

▸ Worked answer

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

Contents

M13-Q3 | Worked answer

Worked reasoning

Known value: (10000,500) gives 20 MPa. Zero force: (0,500) gives 0. Scale test: (20000,500) gives 40. Boundary failure: area 0 is rejected; negative area is also rejected. Add invalid types and nonfinite values only if the shared contract promises their rejection.

Return to question

Contents

[W13] Python usage; MATLAB, C++, Java; language transfer. Review adaptation.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

14 | Integrated engineering review

Connect the entire course in one inspectable report

NAZARBAYEV UNIVERSITY

Start with the contract, not the final plot

- Input: strings with exactly three fields: identifier, numeric value, and exact unit C.
- Accepted values are finite; a nonempty identifier is required.
- Classify accepted readings using inclusive 20–30 °C limits.
- Retain original indices, identifiers, values, and rejection reasons.
- Summarize and rank accepted readings without changing the raw record list.

Important distinction

A valid reading outside the stated interval is an accepted warning. A malformed or incompatible-unit record is rejected and does not become a fabricated zero.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

The raw dataset includes both warnings and rejected records

```
RAW_RECORDS = [
    "T-01,19.5,C", "T-02,20.0,C", "T-03,25.0,C",
    "T-04,30.0,C", "T-05,30.5,C", "T-06,bad,C",
    "T-07,300.0,K",  # Wrong unit for the explicitly stated contract.
]
```

Predict before continuing

Seven raw records enter the pipeline. Decide which records should be accepted, which should warn, and which should be rejected. The C-to-K distinction is a contract issue, not a numeric parsing problem.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

Decompose the pipeline into testable components

Stage	Responsibility	Independent check
Parse	Check structure, identifier, unit, and numeric field.	Preserve T-07 as text.
Validate	Reject nonfinite values.	NaN must not pass as a reading.
Classify	Apply the inclusive temperature interval.	20 and 30 are normal.
Summarize	Use accepted values only.	Mean of five known values.
Rank	Sort complete records by temperature.	IDs remain paired with their values.
Report	Keep both evidence and limitations visible.	Raw = accepted + rejected.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

The parser: structure and identity first

```
# First half of parse_temperature in review_models.py.
def parse_temperature(text):
    """Parse exactly identifier,value,C; no unit conversion."""
    parts = text.split(",")
    if len(parts) != 3:
        raise ValueError("expected three comma-separated fields")
    name, value_text, unit = parts
    name, unit = name.strip(), unit.strip()
    if not name or unit != "C":
        raise ValueError("need a nonempty ID and the exact unit C")
```

Continue on the next page

The next statements convert the value, check finiteness, and return the accepted fields. Both excerpts are parts of the supplied complete function.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

The parser: numerical validity is a separate check

```
# Continuation inside parse_temperature;
    math is imported.
value = float(value_text)
if not math.isfinite(value):
    raise ValueError("temperature must be
    finite")
return name, value
```

Two routes

A malformed numeric string raises ValueError during conversion. A string such as "nan" converts successfully but is rejected by the finite-value check.

What is not inferred

The parser does not guess alternative units, infer calibration, validate a physical sensor, or decide that a high reading is impossible. Those would require additional contracts.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

Keep the failure and success routes explicit

```
# Excerpt: lower and upper are validated, ordered limits.
accepted, rejected = [], []
for index, raw in enumerate(raw_records):
    try:
        name, value = parse_temperature(raw)
    except ValueError as error:
        rejected.append((index, raw, str(error)))
    else:
        category = classify_temperature(value, lower, upper)
        accepted.append((index, name, value, category))
```

Why keep original indices?

After filtering, list position is no longer necessarily the raw-record position. Retaining index and ID allows a warning or failure to be traced back to its source.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

Worked classification: all seven records remain accountable

Raw index	ID / record	Disposition	Reason
0	T-01: 19.5 C	Accepted warning	Below 20.
1	T-02: 20.0 C	Accepted normal	Lower boundary included.
2	T-03: 25.0 C	Accepted normal	Inside the interval.
3	T-04: 30.0 C	Accepted normal	Upper boundary included.
4	T-05: 30.5 C	Accepted warning	Above 30.
5	T-06: bad C	Rejected	Not numerical text.
6	T-07: 300.0 K	Rejected	Wrong unit for the parser.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

A normal mean can hide warnings

Independent calculation

$$\bar{T} = (19.5 + 20 + 25 + 30 + 30.5)/5 = 25.0\ ^\circ\mathrm{C}.$$

What the mean says

The arithmetic average of the five accepted values lies inside the stated interval.

What the mean does not say

Two individual accepted readings are outside the interval. Two additional raw records were rejected. Neither fact disappears because the mean is 25.0 °C.

Empty accepted set

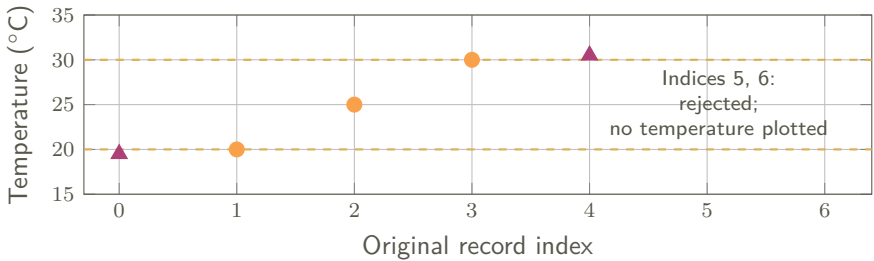
Report absence explicitly, for example mean = None. Do not divide by zero or invent a zero-temperature average.

Contents

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

NAZARBAYEV UNIVERSITY

Graphical evidence retains the warning locations



Read the figure

Dashed lines are the inclusive limits. Triangles are accepted warnings. Rejected records have no plotted y-value; placing them at zero would be misleading.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

Rank records without breaking their associations

```
# Complete processing functions are supplied
      in review_models.py.
from review_models import RAW_RECORDS,
    classify_records
from review_models import temperature_key
accepted, rejected, mean = classify_records(
    RAW_RECORDS)
ranked = sorted(accepted, key=
    temperature_key, reverse=True)
print([record[1] for record in ranked])
print([record[0] for record in rejected])
```

Interpretation

This ranks accepted temperatures from highest to lowest. It does not replace chronological order or imply that a higher reading is more reliable. The raw list remains unchanged.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

Test the entire pipeline as well as the components

```
# End-to-end tests with independent known values.
from review_models import RAW_RECORDS, classify_records
before = RAW_RECORDS.copy()
accepted, rejected, mean = classify_records(RAW_RECORDS)
assert len(accepted) == 5 and len(rejected) == 2
assert len(accepted) + len(rejected) == len(RAW_RECORDS)
assert mean == 25.0
assert [r[0] for r in accepted if r[3] != "normal"] == [0, 4]
assert RAW_RECORDS == before
```

Additional cases

Test an empty input list, all rejected records, repeated identifiers, exact limits, malformed field counts, and nonfinite values. State which behavior is promised before writing a test.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

Explain the cost of the complete report

Parsing and classification

For n records of bounded length, a single pass has $\Theta(n)$ work. If record lengths vary, total text length also matters.

Summary and ranking

Summary is linear in accepted count k. A general comparison sort has an $O(k \log k)$ worst-case bound. Retained accepted and rejected records require linear storage.

Do not drop the explanation

A fast program can still misinterpret units, erase rejected records, or produce a misleading plot. Correctness and meaning remain requirements.

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

NAZARBAYEV UNIVERSITY

INT-Q1 | Account for every record

Try before viewing the solution

For the displayed seven-record dataset, report the accepted count, rejected count, normal count, warning count, and mean. Explain why the warning count is not the rejected count even though both happen to equal two.

▸ Worked answer

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

INT-Q1 | Worked answer

Worked reasoning

Accepted = 5; rejected = 2; normal = 3; warnings among accepted = 2; mean = 25.0 °C. A warning is a valid finite reading outside the stated operating interval. Rejection means the record fails the parsing or validity contract. The equal counts are incidental.

Return to question

Contents

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

NAZARBAYEV UNIVERSITY

INT-Q2 | Choose a cross-language regression set

Try before viewing the solution

Suppose the complete report is translated to another language. Which outputs and side effects must be compared before claiming that the translation preserves behavior?

▸ Worked answer

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

Contents

INT-Q2 | Worked answer

Worked reasoning

Compare accepted records and original indices, classifications at exact limits, rejected indices and meaningful reasons, mean with a stated tolerance, ranking and tie policy, and raw-data preservation. Also test empty and all-rejected inputs. Similar-looking source or agreement only on the mean is insufficient.

Return to question

Contents

[R14] existing Week 14 temperature-review adaptation of [W03], [W07], [W13]. Synthetic data.

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

15 | Mixed-topic practice

Attempt each question before opening its worked answer

NAZARBAYEV UNIVERSITY

X01 | Choose the strongest correctness claim

Try before viewing the solution

1. A: The program runs, so its engineering model is valid.
2. B: Passing selected tests proves correctness for every input.
3. C: The result satisfies selected independent checks under stated assumptions.
4. D: More decimal places prove greater physical accuracy.

▸ Worked answer

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

X01 | Worked answer

Worked reasoning

C is the defensible statement. Execution alone does not validate the model. A finite test suite may miss cases or contain wrong expectations. Numerical formatting does not improve the quality of the data or model.

Return to question

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

NAZARBAYEV UNIVERSITY

X02 | A default parameter and a local name

```
# Same spelling does not imply one shared
    variable.
gain = 10
def scale(value, gain=2):
    return value * gain
print(scale(3), scale(3, gain=4))
print(gain)
```

Write your prediction first

1. Write both results.
2. Which value is global and which is local?
3. Does the keyword argument change the default definition?

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X02 | Worked answer

```
6 12
10
```

Why this happens

The default call uses gain = 2 and returns 6; the explicit keyword call uses 4 and returns 12. Neither changes the separate outer gain = 10. A supplied argument overrides the default for that call, not permanently.

Return to question

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

NAZARBAYEV UNIVERSITY

X03 | A tuple can contain a mutable object

```
# Tuple immutability does not make contained
      lists immutable.
record = ("A", [1, 2])
record[1].append(3)
print(record)
```

Write your prediction first

1. Predict the printed tuple.
2. Which object changed?
3. Would assigning record[0] = "B" be permitted?

▸ Worked answer

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

X03 | Worked answer

```
('A', [1, 2, 3])
```

Why this happens

The contained list changes, producing ("A", [1, 2, 3]). The tuple's slots are unchanged. Assigning to record[0] would raise TypeError because tuple slots cannot be replaced.

Return to question

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

NAZARBAYEV UNIVERSITY

X04 | Select an input-space guarantee

Try before viewing the solution

1. A: The iteration counter reached its maximum.
2. B: The displayed residual has six decimal places.
3. C: A continuous root remains in a bracket of width 0.002, and the reported estimate is its midpoint.
4. D: Two adjacent guesses rounded to the same displayed value.

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X04 | Worked answer

Worked reasoning

C gives an input-space midpoint error bound of 0.001 in exact arithmetic. The other observations alone do not establish that error bound. State the input unit and consider floating-point limitations when interpreting the guarantee.

Return to question

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

NAZARBAYEV UNIVERSITY

X05 | Repair invalid data without falsifying it

Try before viewing the solution

A program catches every conversion exception and substitutes 0.0. Its average then lies inside the normal interval. Explain the defect and propose a report that retains the failed record.

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

NAZARBAYEV UNIVERSITY

X05 | Worked answer

Worked reasoning

The replacement invents a measured value and can distort both the average and classifications. Catch the expected conversion failure specifically, retain its original index, raw text, and reason, and exclude it according to a declared policy. Report accepted and rejected counts with the summary.

Return to question

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

NAZARBAYEV UNIVERSITY

X06 | Return value versus shared state

```
# Aliasing and method return values are
    separate questions.
from intro_sensor import IntroSensor
a = IntroSensor("A", "deg C", 25.0, 20.0,
    30.0)
b = a
answer = b.update(35.0)
print(a.value, b.value, answer)
```

Write your prediction first

1. Predict the three printed values.
2. Why does b.value change?
3. Why is answer None?

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X06 | Worked answer

```
35.0 35.0 None
```

Why this happens

a and b refer to the same sensor. Updating via b changes the single object to 35.0. The update method has no value-returning statement, so answer is None.

Return to question

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

NAZARBAYEV UNIVERSITY

X07 | Classify a hidden nested scan

Try before viewing the solution

An existing list of n records is traversed once. On each pass, a method recomputes the sum of all n readings. Which is the tight time class under bounded-cost arithmetic: $\Theta(1)$, $\Theta(\log n)$, $\Theta(n)$, or $\Theta(n^2)$? Explain.

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X07 | Worked answer

Worked reasoning

$\Theta(n^2)$: each of n outer passes triggers an n-item scan. A short method call hides the inner work. Reusing a single subtotal can reduce work only when the data and required result remain unchanged.

Return to question

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

NAZARBAYEV UNIVERSITY

X08 | An insertion position needs a bounds check

```
# Insertion positions can lie one place
    beyond the last item.
from bisect import bisect_left
values = [10, 20, 30]
pos = bisect_left(values, 40)
found = pos < len(values) and values[pos] ==
    40
print(pos)
print(found)
```

Write your prediction first

1. Predict both outputs.
2. Why is values[pos] not evaluated here?
3. What would accessing it without the guard do?

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X08 | Worked answer

```
3
False
```

Why this happens

The insertion position is 3, equal to the list length. The first condition is false, so and short-circuits and avoids indexing. Accessing values[3] would raise IndexError; an insertion slot need not be an existing element.

Return to question

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

NAZARBAYEV UNIVERSITY

X09 | One lookup versus many lookups

Try before viewing the solution

You have one target and an unsorted list. A classmate proposes sorting it before binary search because binary search is logarithmic. What is missing from the argument? When could preprocessing become worthwhile?

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X09 | Worked answer

Worked reasoning

The argument omits sorting cost and any index-preservation requirements. One linear scan is $O(n)$; general sorting first usually adds $O(n \log n)$ work. Repeated queries on largely unchanged data may amortize the preparation, but include updates, memory, and the required search contract.

Return to question

Contents

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

NAZARBAYEV UNIVERSITY

X10 | Choose the right interpretation of a failed update

Try before viewing the solution

1. A: Every out-of-limit reading must be discarded.
2. B: A rejected invalid proposal should leave the previous valid state unchanged when the setter promises that contract.
3. C: A setter returning None did not perform an update.
4. D: A leading underscore prevents all external access.

▸ Worked answer

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

X10 | Worked answer

Worked reasoning

B expresses the stated failure contract. A valid out-of-limit reading should generally be retained and flagged under this monitoring model. A setter can mutate state and return None. A leading underscore signals a convention rather than enforced security.

Return to question

[R] Original cumulative practice using the supplied course topics. Not announced examination questions.

Contents

NAZARBAYEV UNIVERSITY

SELF-STUDY REVIEW

16 | Final checks and reference

Use your mistakes to choose the next revision target

NAZARBAYEV UNIVERSITY

Readiness means being able to explain and verify

- Trace a program with a new input and explain every change in state.
- Write or repair a focused function with a clear contract.
- Select tests that expose boundaries, missing data, and unintended mutation.
- Explain why a loop or recursion terminates and whether it succeeded.
- Justify algorithm cost and preconditions; preserve meaning across languages.

Not a score prediction

These practice tasks are formative. They do not announce examination questions, the examination format, permitted resources, or a grading threshold.

Contents

[C] instructor-provided scope; [W01]–[W13] supplied course materials. Review synthesis.

NAZARBAYEV UNIVERSITY

Use an error log instead of rereading everything equally

Record	Example
Exercise ID	M05-Q2: aliasing.
My prediction	I expected raw to remain unchanged.
Observed / worked result	raw changed through alias.
Underlying misconception	I treated assignment as a copy.
Repair and retest	Draw references; try a new flat list and then a nested list.

Return with a changed example

A new input tests whether you understand the rule rather than remember the previous answer. The contents links and PDF bookmarks make targeted review easier.

Contents

[C] instructor-provided scope; [W01]–[W13] supplied course materials. Review synthesis.

NAZARBAYEV UNIVERSITY

Source basis and the scope of this review

- The instructor's fourteen-week outline controls topic order and coverage.
- The supplied Week 1–13 decks and earlier Week 14 review provide terminology, conventions, and recurring engineering examples.
- The course manuscript supplies the broader computation–evidence–interpretation framing. Its alternative chapter map does not replace the actual course sequence.
- Exercises and additional explanations in this revision are original self-study adaptations, not quotations or announced examination questions.

What changed from the previous Week 14 deck

The three-lecture schedule, timed activities, pair-work directions, and instructor-only answer dependence have been replaced by topic modules and visible question–solution pairs.

Contents

[C] instructor-provided scope; [W01]–[W13] supplied course materials. Review synthesis.

NAZARBAYEV UNIVERSITY

Explicit clarifications and code status

- The Week 3 `"PUMP-107A"[::2]` result is corrected openly to `"PM-0A"`.
- Fibonacci keeps $F(0) = F(1) = 1$; the complete F(4) tree shows nine calls.
- The simple IntroSensor and the checked review Sensor have separate names and assumptions.
- Patterns and excerpts refer to the complete supplied files; runnable examples are identified in the README.

Language checks

C++ and Java division checks are executed. MATLAB examples are reading exercises based on Week 13; no MATLAB execution is claimed.

Contents

[C] instructor-provided scope; [W01]–[W13] supplied course materials. Review synthesis.

NAZARBAYEV UNIVERSITY

Source reference key

Key	Material
[C]	ENG 101 specification, Fall 2026; instructor-provided sequence.
[W01]–[W06]	Supplied complete Week 1–6 decks.
[W07]–[W08]	Supplied testing/debugging and OOP decks.
[W09]–[W13]	Prior decks: inheritance, efficiency, search/sort, language transfer.
[R14] / [R]	Prior Week 14 examples and new self-study adaptations.
Manuscript	Y. Wang, Programming for Engineers with Python, v20260815.

The course in one sentence

A trustworthy result needs a clear question, meaningful data, a justified method, tested code, and an honest interpretation.

Contents

[C] instructor-provided scope; [W01]–[W13] supplied course materials. Review synthesis.